

物件導向程式設計課程專題書面報告

個人記帳管理系統

Personal Budget Book

第 13 組 | 自選題

組員 簡俞捷 B13204013 鍾秉叡 B13204043 王獻霆 R13945041

課程 物件導向程式設計

繳交日期 2026 年 6 月 21 日

GitHub Repository

<https://github.com/zxc29051617/OOP-G13-Personal-Budget-Book>

摘要

本專題實作一套以 Java Swing 開發的桌面版個人記帳管理系統。系統將現金、銀行帳戶與電子支付等不同資金來源整合在同一個應用程式中，提供收入與支出記錄、錢包管理、月預算、分類統計、歷史紀錄及簡單查詢助手等功能。使用者完成每一筆記帳時，系統會將該筆紀錄視為替當月小樹澆水一次；當月累積 20 筆記帳即可完成一棵樹，以遊戲化方式鼓勵持續記帳。

程式採用簡單分層架構，將 app、model、ui、service 與 persistence 分開管理。資料主要儲存在 MySQL 8.0，Java 程式透過 JDBC 與 MySQL Connector/J 執行 SQL；FlatLaf 則用於改善 Swing 預設外觀。查詢助手以關鍵字規則配合目前資料庫統計結果回答常見問題，不使用外部生成式 AI API，也不屬於真正的 RAG 系統。

關鍵字：Java Swing、MySQL、JDBC、物件導向、個人記帳、資料視覺化

一、專題背景與動機

日常消費可能分散在現金、銀行帳戶與電子支付等不同來源。如果只依靠記憶或單一帳戶紀錄，使用者不容易掌握整體收支，也難以快速判斷本月是否接近預算上限。因此，本專題選擇「個人記帳管理系統」作為自選題，將多錢包管理、收支統計、預算追蹤與歷史紀錄整合至同一個桌面應用程式。

本系統使用 Swing 建立圖形化介面，並參考行動版 Finance App 的資訊層級與操作方式。使用者可以透過底部導覽列切換首頁、記帳、分析、錢包與紀錄頁面，並從右上角開啟設定或查詢助手。這項設計同時涵蓋課程中的類別與物件、封裝、繼承、列舉、例外處理、GUI 事件處理及資料庫持久化等練習。

二、專案目標

- 整合多種資金來源，集中管理現金、銀行帳戶與電子支付等錢包。
- 提供快速記帳流程，記錄日期、時間、收入或支出、分類、錢包、金額與備註。
- 即時呈現總餘額、本月收入、本月支出、收支差額及預算使用率。
- 提供歷史交易查詢與刪除功能，方便使用者維護記帳資料。
- 以分類長條圖呈現本月支出分布，協助辨識主要花費。
- 透過每筆記帳澆水一次、每月 20 次完成一棵樹的方式增加持續使用動機。
- 使用 MySQL 與 JDBC 建立可持久保存、具關聯約束的資料儲存機制。
- 以簡單查詢助手提供常見財務問題的快速回答。
- 運用物件導向與分層設計，使畫面、邏輯與資料庫操作各自負責明確工作。

三、系統功能總覽

功能模組	主要內容	對應程式
首頁總覽	顯示總餘額、本月收入、本月支出、預算使用率、小樹狀態與最近 5 筆交易。	ui.HomePanel
快速記帳	輸入交易類型、日期、時間、分類、錢包、金額與備註，儲存後替小樹澆水一次。	ui.QuickEntryPanel

功能模組	主要內容	對應程式
財務分析	彙整本月收入、支出與結餘，並繪製分類支出長條圖。	ui.AnalysisPanel、 service.StatsService
錢包管理	新增或刪除錢包，查看各錢包初始金額及目前餘額。	ui.WalletPanel、 model.Account
交易紀錄	依日期與時間顯示所有交易，並支援刪除紀錄。	ui.HistoryPanel、 model.Transaction
設定與種樹	設定每月支出預算，查看本月澆水次數、進度及已完成樹木數。	ui.SettingsPanel、 ui.TreeGardenPanel
查詢助手	以常用問題按鈕或文字輸入，查詢支出、預算、錢包、樹木與最近交易。	ui.AssistantPanel、 service.QueryAssistan tService
資料持久化	使用 JDBC 將錢包、交易及設定保存至 MySQL。	persistence.BudgetBo okStore

四、開發環境與使用技術

項目	內容
程式語言	Java 25
圖形化介面	Java Swing
介面外觀	FlatLaf 3.7.1
資料庫	MySQL Server 8.0
資料庫連線	JDBC、MySQL Connector/J
資料處理	Java Collection、LocalDate、LocalTime、YearMonth
編譯與執行	run.bat，或使用 javac / java 指令
版本控制	Git、GitHub Public Repository

專案的第三方 jar 放置於 lib 資料夾，但 lib/*.jar 已被 Git 忽略，因此從 GitHub 下載後，需自行加入 MySQL Connector/J 與 FlatLaf。MySQL 密碼則存放在本機的 data/database.properties，該資料夾同樣不會上傳至 GitHub，以避免帳號密碼外洩。

五、系統架構設計

本專案採用簡單分層架構，而不宣稱為完整 MVC。主視窗仍直接管理部分頁面切換與互動流程，但資料模型、統計邏輯、查詢邏輯、資料庫操作及介面已依套件分離，使各類別的責任較為明確。

Personal Budget Book 系統分層架構

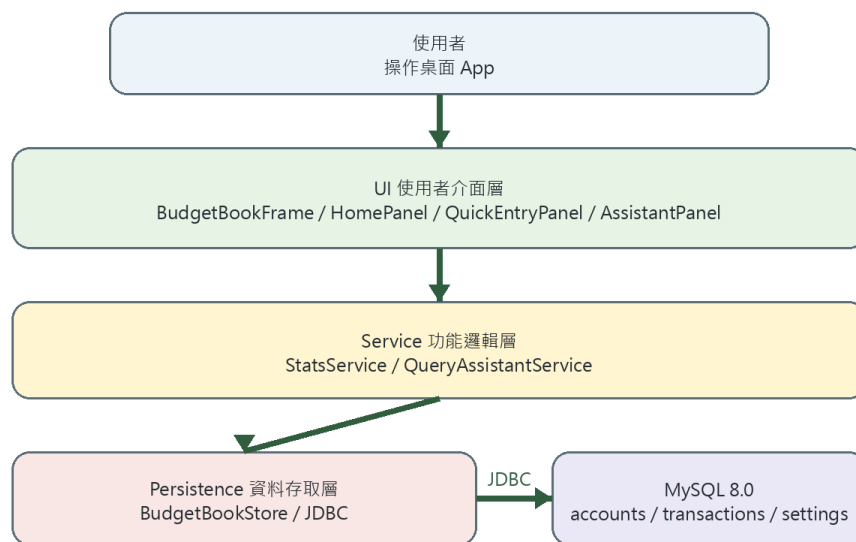


圖 1 系統分層架構

層級	主要職責	主要類別
App	啟動程式、設定 FlatLaf、建立資料存取物件與主視窗。	BudgetBookApp
UI	顯示畫面、接收操作、切換頁面及呈現資料。	BudgetBookFrame、各功能 Panel

層級	主要職責	主要類別
Service	計算財務統計、種樹進度及關鍵字查詢回答。	StatsService、 QueryAssistantService
Persistence	連線 MySQL、建立資料表及執行新增、查詢、刪除與設定更新。	BudgetBookStore
Model	定義錢包、交易與系統設定的資料格式。	Account、Transaction、 BudgetSettings

六、程式啟動流程

程式入口為 `app.BudgetBookApp`。main 方法透過 `SwingUtilities.invokeLater` 將 GUI 建立工作放到 Swing 事件派發執行緒中，避免直接在主執行緒操作 Swing 元件。啟動流程如下：

1. 執行 `app.BudgetBookApp.main`。
2. 嘗試載入 `FlatLightLaf`；若套件不可用，改用系統 `Look and Feel`。
3. 建立 `BudgetBookStore(Paths.get("data"))`。
4. 從 `data/database.properties` 或環境變數取得 MySQL 連線設定。
5. 連線 MySQL，確認 `accounts`、`transactions` 與 `settings` 資料表存在。

6. 讀取錢包、交易與設定資料；如果資料庫為空且存在舊版 CSV，則執行一次性匯入。
7. 若資料庫仍沒有錢包，建立現金、銀行帳戶與電子支付三個預設錢包。
8. 建立 BudgetBookFrame 並顯示主視窗。

七、資料模型設計

Model 層只負責描述資料格式，不直接處理 Swing 畫面或 SQL。核心資料模型如下：

類別	重要欄位	說明
Account	id、name、type、initialBalance	代表一個錢包，例如現金、銀行帳戶或電子支付。
Transaction	id、date、time、kind、category、accountId、amount、note	代表一筆收入或支出，並透過 accountId 關聯錢包。
BudgetSettings	monthlyLimit、points	儲存每月支出預算；points 為舊版相容保留欄位。

Transaction.Kind 使用 enum 將交易類型限制為 INCOME 或 EXPENSE，比直接以任意字串表示更安全。Transaction 同時保存 LocalDate 與 LocalTime，使同一天的交易仍可依實際時間排序。

八、資料庫設計

系統主要資料儲存使用 MySQL，預設資料庫名稱為 `personal_budget_book`，字元集使用 `utf8mb4`，排序規則為 `utf8mb4_unicode_ci`，以支援繁體中文內容。程式啟動時會自動建立資料表，也可使用 `db/schema.sql` 手動建置。

圖 2 資料表關聯與主要欄位

8.1 accounts 資料表

欄位	型態 / 限制	用途
id	INT、PK、 AUTO_INCREMENT	錢包唯一識別碼。
name	VARCHAR(100)、NOT NULL	錢包名稱。
type	VARCHAR(100)、NOT NULL	錢包類型。
initial_balance	DOUBLE、NOT NULL	錢包建立時的初始金額。

8.2 transactions 資料表

欄位	型態 / 限制	用途
id	INT、PK、 AUTO_INCREMENT	交易唯一識別碼。
date	DATE、NOT NULL	交易日期。
transaction_time	TIME、NOT NULL	交易時間；舊資料預設為 00:00:00。
kind	VARCHAR(20)、NOT NULL	INCOME 或 EXPENSE。
category	VARCHAR(100)、NOT NULL	收入或支出分類。
account_id	INT、FK、NOT NULL	對應 accounts.id。
amount	DOUBLE、NOT NULL	交易金額。
note	VARCHAR(255)	使用者備註，可為空值。

transactions.account_id 是外鍵，對應 accounts.id，形成一個錢包對多筆交易的一對多關係。外鍵採用 ON DELETE RESTRICT，避免刪除仍有交易紀錄的錢包，造成交易失去來源。

8.3 settings 資料表

欄位	型態 / 限制	用途
id	INT、PK	系統設定識別碼，目前固定使用 1。
monthly_limit	DOUBLE、NOT NULL	每月支出預算。
points	INT、NOT NULL	舊版相容保留欄位，目前種樹進度不依賴此欄位。

九、資料存取與持久化設計

BudgetBookStore 是資料存取核心。它透過 DriverManager 建立 JDBC Connection，使用 PreparedStatement 執行參數化 SQL，以避免手動拼接輸入資料。錢包與交易的新增、刪除會立即寫入 MySQL；設定資料則由 save() 更新至 settings 資料表。

操作	主要 SQL / 方法	處理方式
載入資料	SELECT	啟動時讀取三張資料表並建立 Java 物件。
新增錢包	INSERT INTO accounts	取得自動產生的 id 後加入記憶體清單。
新增交易	INSERT INTO transactions	儲存日期、時間、類型、分類、錢包、金額與備註。
刪除交易	DELETE FROM	依交易 id 刪除。

操作	主要 SQL / 方法	處理方式
	transactions	
刪除錢包	DELETE FROM accounts	若錢包已有交易，程式與外鍵約束都會阻止刪除。
更新設定	UPDATE settings	儲存 monthly_limit 與保留的 points 欄位。

為保留舊版本資料，BudgetBookStore 仍包含 CsvUtil 與 CSV 匯入程式。只有在 MySQL accounts 資料表為空且 data 資料夾存在舊 CSV 時才會匯入；匯入完成後，日常讀寫均以 MySQL 為主。

十、統計與查詢助手邏輯

10.1 StatsService

方法	功能
totalBalance()	計算所有錢包的目前餘額後加總。
monthlyIncome(month)	統計指定月份的收入總額。
monthlyExpense(month)	統計指定月份的支出總額。

方法	功能
monthlyExpenseByCategory(month)	依分類彙整指定月份的支出。
monthlyWaterCount(month)	計算指定月份的記帳筆數，作為澆水次數。
monthlyTreeStatus(month)	依 0 筆、1 至 19 筆及 20 筆以上回傳枯萎、成長中或完成一棵樹。
completedTreeCount()	計算歷史月份中達成 20 筆記帳的月份數。

10.2 QueryAssistantService

查詢助手接收使用者輸入後，以關鍵字判斷欲查詢的項目，再呼叫 StatsService 或 BudgetBookStore 取得結果。可回答本月支出、本月收入、剩餘預算、總餘額、錢包數、小樹狀態、最近交易及最大支出分類等問題。

設計限制：目前助手是規則式關鍵字查詢，不會呼叫外部大型語言模型，也沒有向量資料庫、文件切分、Embedding 或檢索生成流程，因此不屬於真正的 RAG。

十一、使用者介面設計

主視窗 BudgetBookFrame 使用 JFrame 建立應用程式外框，主要內容區以 CardLayout 切換不同 RefreshablePanel。畫面採用接近手機 Finance App 的窄版配置，底部固定放置首頁、記帳、分析、錢包與紀錄五個導覽項目；設定與查詢助手由頁面右上角進入。

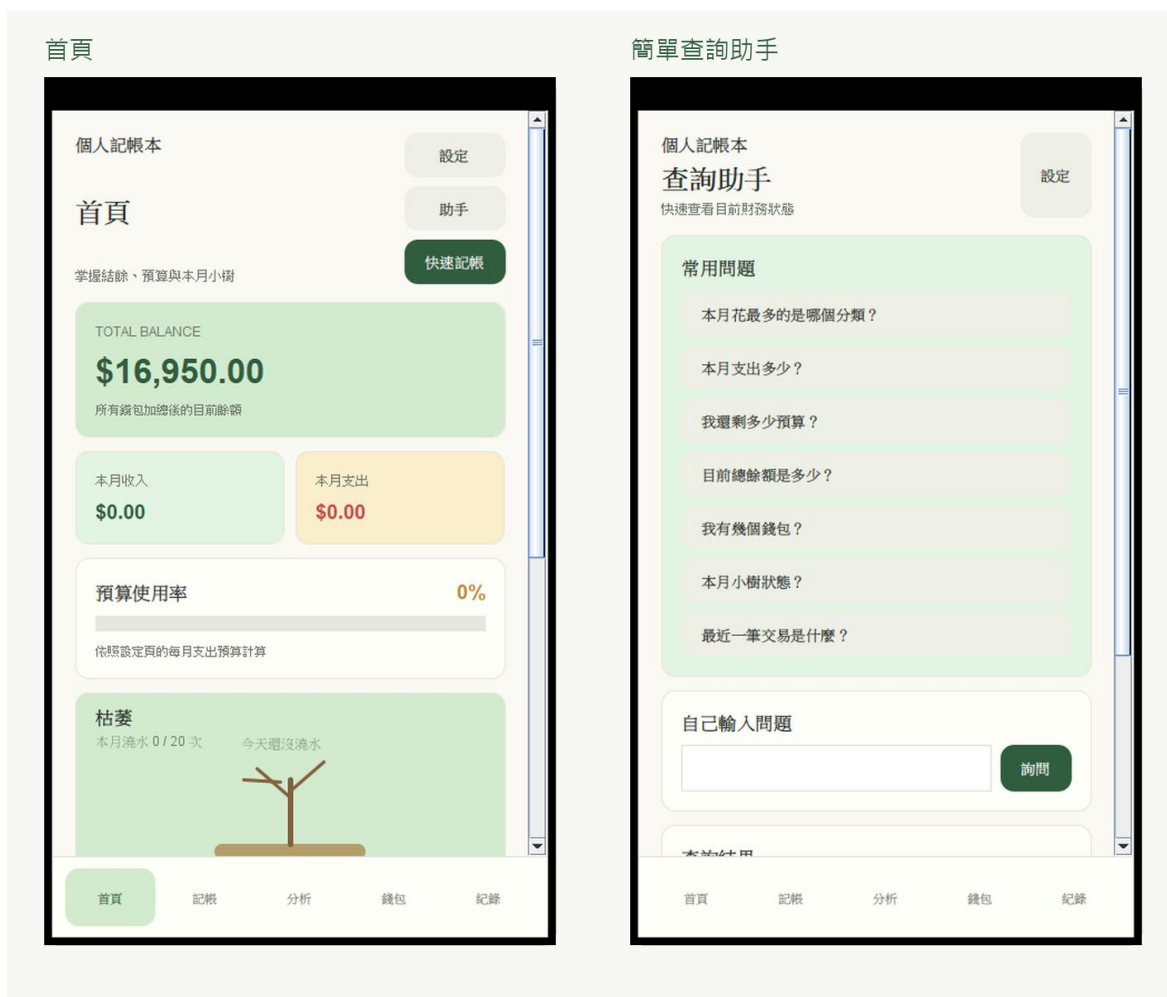


圖 3 首頁與簡單查詢助手介面

- 首頁以卡片呈現總餘額、本月收支、預算使用率、小樹狀態與最近交易。
- 快速記帳使用下拉選單與輸入欄位，限制金額必須大於 0，並要求使用者選擇錢包。
- 財務分析頁以自訂 JPanel 和 Graphics2D 繪製分類長條圖。
- 錢包與歷史紀錄採卡片式列表，提供刪除操作與錯誤提示。
- Ui 類別集中管理色彩、字型、圓角卡片、按鈕與輸入元件樣式。

- FlatLaf 用於改善 Swing 原生元件外觀；若未載入成功，程式仍可使用系統 Look and Feel。

十二、物件導向概念應用

OOP 概念	本專案中的應用
封裝 Encapsulation	Account、Transaction 與 BudgetSettings 將欄位設為 private，透過方法控制存取。
類別與物件	每個錢包、交易、設定及 Swing 頁面都是獨立物件。
列舉 Enum	Transaction.Kind 將交易種類限制為 INCOME 或 EXPENSE。
繼承 Inheritance	各功能頁面繼承 RefreshablePanel，統一實作 refresh()。
多型 Polymorphism	BudgetBookFrame 以 RefreshablePanel 型別管理不同頁面並統一刷新。
組合 Composition	BudgetBookFrame 持有 BudgetBookStore、StatsService 及多個頁面物件。
分層與單一責任	model、service、persistence、ui 分別處理資料、邏輯、儲存與畫面。
例外處理	啟動、輸入驗證、資料庫連線及新增刪除操作均使用例外處理與提示視窗。

十三、主要使用流程

13.1 新增記帳流程

1. 使用者進入快速記帳頁面。
2. 選擇支出或收入，輸入日期、時間、分類、錢包、金額與備註。
3. 程式檢查是否已選擇錢包、金額是否大於 0，以及日期與時間格式是否正確。
4. QuickEntryPanel 呼叫 BudgetBookStore.addTransaction()。
5. BudgetBookStore 使用 PreparedStatement 將資料 INSERT 至 transactions。
6. 儲存成功後刷新所有頁面，首頁、分析、錢包及歷史紀錄立即顯示最新狀態。
7. 本月記帳筆數增加一筆，種樹介面顯示澆水進度。

13.2 查詢助手流程

1. 使用者點選常用問題，或在輸入欄自行輸入問題。
2. AssistantPanel 將問題傳給 QueryAssistantService。
3. 服務依「本月支出」、「預算」、「錢包」、「小樹」等關鍵字判斷查詢類型。
4. 服務呼叫 StatsService 或 BudgetBookStore 取得目前資料。
5. 將結果格式化為繁體中文文字並顯示於查詢結果區。

十四、測試與驗證

專案以編譯、MySQL 連線、功能操作與畫面檢查驗證主要流程。下列項目可作為展示及繳交前的驗收清單：

測試項目	操作方式	通過條件
編譯	執行 run.bat 的 javac 指令。	所有套件成功編譯至 out 資料夾。
MySQL 連線	啟動 MySQL 後執行程式。	成功載入資料並顯示主視窗。
首次啟動	使用空資料庫啟動。	自動建立三張表、設定列及三個預設錢包。
新增支出	輸入合法日期、時間、錢包與正數金額。	資料寫入 transactions，頁面統計同步更新。
輸入驗證	輸入 0、負數或錯誤日期格式。	顯示錯誤訊息且不新增交易。
刪除已使用錢包	刪除已有交易的錢包。	程式與外鍵限制阻止刪除。
分類分析	新增多種分類支出。	分析頁顯示各分類金額與長條圖。
種樹進度	建立不同數量的當月交易。	0 筆為枯萎、1 至 19 筆成長中、20 筆以上完成一棵樹。
查詢助手	詢問本月支出、錢包數或最近交	顯示與目前資料一致的回答。

測試項目	操作方式	通過條件
	易。	
資料持久化	新增資料後關閉並重新啟動。	MySQL 中的資料仍能重新載入。

十五、分工表

下表保留待填欄位，請依實際組員姓名、學號與工作內容調整；所有貢獻度加總必須為 100%。

組員	主要分工	具體工作內容	貢獻度
組員 A [姓名 / 學 號]	專案整合	架構規劃、MySQL/JDBC 整合、主程式 與 GitHub 管理。	___%
組員 B [姓名 / 學 號]	介面設計	Swing 頁面、FlatLaf、導覽、卡片與圖 表介面。	___%
組員 C [姓名 / 學 號]	功能與文件	資料模型、統計、查詢助手、測試、 PPT 與 Report。	___%

十六、AI 使用說明

本專題開發過程中使用 Codex 與 ChatGPT 作為輔助工具。AI 用於程式架構整理、Swing 介面設計建議、MySQL/JDBC 遷移、錯誤分析、測試方向及文件撰寫；所有內容仍由組員確認、修改、執行與測試後才納入專案。

使用面向	具體使用方式
架構整理	協助整理 app、model、persistence、service、ui 分層與類別責任。
程式設計輔助	提供 Swing 介面、JDBC、PreparedStatement、統計與查詢助手的實作方向。
錯誤修正	分析編譯錯誤、MySQL 連線、日期時間解析與畫面刷新問題。
文件整理	協助 README、資料庫設計、程式架構、PPT 與書面報告文字。

- AI 僅作為輔助工具，不作為唯一開發來源。
- AI 建議的程式碼必須由組員理解後才使用。
- 所有程式修改均需實際編譯、執行與測試。
- 報告內容必須符合目前專案，不誇大查詢助手為生成式 AI 或 RAG。

- 若 AI 建議與課程要求或實際程式不符，以課程要求與實作內容為準。

十七、限制與未來改進

- 目前為本機單一使用者桌面程式，尚未提供登入、權限或多使用者資料隔離。
- 查詢助手採固定關鍵字規則，無法理解複雜語意；未來可加入自然語言解析，若有文件知識需求再評估真正 RAG。
- 金額目前使用 `double`，未來可改用 `BigDecimal` 提升財務計算精準度。
- 交易分類為程式內固定陣列，未來可新增 `category` 資料表，讓使用者自訂分類。
- 目前依賴本機 MySQL Server 與外部 jar，未來可提供安裝包、Docker Compose 或內嵌資料庫版本降低部署門檻。
- 刪除操作目前不可復原，未來可加入確認視窗、軟刪除或操作復原。
- 分析頁目前聚焦本月分類支出，未來可增加月趨勢、年度比較、圓餅圖及資料匯出。
- `settings.points` 為舊版保留欄位，未來可在資料庫遷移後移除，讓結構更精簡。

十八、結論

本專題完成一套以 Java Swing 與 MySQL 實作的個人記帳管理系統，涵蓋多錢包管理、快速記帳、日期時間紀錄、即時財務統計、分類圖表、月預算、歷史紀錄、種樹激勵及簡單查詢助手。系統透過 JDBC 將資料持久保存於 MySQL，並以外鍵維持錢包與交易之間的一致性。

從物件導向角度來看，專案實際應用了封裝、類別與物件、列舉、繼承、多型、組合與例外處理。分層架構讓 UI、統計邏輯、查詢邏輯與資料庫操作得以分開維護，也使後續加入新統計方法、改良查詢助手或調整資料庫結構時，能在較清楚的責任範圍內修改。

參考資料

- <https://www.elle.com/tw/life/tech/g70140471/spending-app-2026/>
- <https://doc.moze.app/>
- <https://blog.aifian.com/2025/05/05/expense-tracker-app/>
- 課程專題說明文件：物件導向程式設計課程專題要求。

<https://github.com/zxc29051617/OOP-G13-Personal-Budget-Book>

附錄 A：執行環境與重現流程

助教從 GitHub 下載專案後，仍需準備 Java、MySQL 及兩個 jar。建議依下列流程重現：

1. 確認已安裝 Java 25 與 MySQL Server 8.0，並啟動 MySQL 服務。
2. 將 mysql-connector-j-*.jar 與 flatlaf-*.jar 放入專案 lib 資料夾。
3. 建立 data/database.properties，填入本機 MySQL 帳號與密碼。
4. 在 PowerShell 進入專案根目錄並執行 .\run.bat。
5. 程式會建立 personal_budget_book 資料庫所需的三張資料表並開啟主視窗。

```
url=jdbc:mysql://localhost:3306/personal_budget_book?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=Asia/Taipei&createDatabaseIfNotExist=true&useUnicode=true&characterEncoding=utf8
```

```
user=root  
password=你的 MySQL 密碼  
  
cd C:\專案路徑\PersonalBudgetBook  
.\run.bat
```